

LPU Backend Case Studies 2026

Plus Points in Implementation (Overall Evaluation Criteria)

1. Authentication:

- Implement robust user authentication protocols to ensure secure access.

2. Cost Estimation - Time and Space:

- Conduct a thorough analysis of time and space complexity in the system.
- Utilize efficient algorithms and data structures to optimize both time and space requirements.

3. Handling System Failure Cases:

- Implement fault-tolerant mechanisms to address system failures.
- Employ backup and recovery strategies for data integrity.
- Develop comprehensive error recovery procedures to minimize downtime.

4. Object-Oriented Programming Language (OOPS):

- Choose a robust OOPS language for structured and modular code.
- Leverage OOPS principles such as encapsulation, inheritance, and polymorphism for maintainability and extensibility.

5. Trade-offs in the System:

- Clearly define and document trade-offs made during system design.
- Evaluate and communicate the rationale behind architectural and design decisions.
- Consider trade-offs in terms of performance, scalability, and maintainability.

6. System Monitoring:

- Implement comprehensive monitoring tools to track system performance.
- Utilize real-time dashboards and logging mechanisms to promptly identify and address issues.

7. Caching:

- Integrate caching mechanisms to enhance system response times.
- Utilize caching for frequently accessed data to reduce database load.
- Implement cache eviction policies for optimal resource utilization.

8. Error and Exception Handling:

- Develop a robust error and exception handling framework.

- Provide meaningful error messages for effective debugging.
 - Regularly review and update error-handling strategies based on system usage patterns.
-

Instructions

1. Read and Understand the Problem Statement:

- Carefully read the case study you pick. Understand the requirements, the inputs, the expected outputs, and any rules mentioned.

2. Choose a Programming Language:

- Select a language you are comfortable with. An object-oriented language (Java, C#, Python, etc.) is preferred.

3. Design Your Solution:

- Plan the structure first. Decide your database tables (ER diagram), the main classes, and the APIs you will build before writing code.

4. Write the Code:

- Write clean code with meaningful names and proper layering (controller / service / database).
- Break the problem into small functions and modules so it is easy to read and test.

5. Test Your Code:

- Test with normal inputs AND edge cases (empty data, wrong input, duplicate requests).
- Add a few unit tests for the important logic.

6. Document Your Code:

- Add a README: how to run the project, what assumptions you made, and a short note on the choices/trade-offs you took.

7. Submit Your Solution:

- Push your code to a public GitHub repository and share the link.
- Keep a meaningful commit history – do not upload everything in one commit.

8. Demonstration:

- Add a short demo video or screenshots showing the main features and a few sample API calls (Postman / curl).

3. Office Shuttle – Segment-Based Seat Booking

An office shuttle is a bus that runs a fixed route with several stops – $A \rightarrow B \rightarrow C \rightarrow D$ – at fixed times, and employees book a seat for the part of the journey they need. The clever bit: one physical seat can be sold to different people on different parts of the route, as long as they are not on the bus at the same time. So seat #5 can carry one person from $A \rightarrow B$ and a completely different person from $B \rightarrow D$. “Is a seat free?” is therefore not a yes/no flag – it is an overlap question. Getting this right is the whole challenge (and it is exactly what makes shuttle booking harder than a normal bus ticket).

I. Routes, Stops & Trips

Set up a shuttle route as an ordered list of stops with timings, and a trip that runs it on a given day with a fixed number of seats.

-  **Route & Stops** – A route has ordered stops with arrival times (A 10:00, B 10:15, C 10:30, D 10:45).
-  **Trip Capacity** – Each trip has a seat count (e.g., 40). A passenger books a segment: from one stop to a later stop.

II. Segment-Based Seat Availability (the crown-jewel problem)

A seat is free for a requested segment only if no existing booking on that seat overlaps it. Two segments overlap if they share any part of the road – not if they merely touch at a stop.

-  **Overlap Rule** – Seat S can take segment $[Y_s, Y_d]$ only if every booking $[X_s, X_d]$ on S satisfies $Y_d \leq X_s$ OR $Y_s \geq X_d$ (i.e., they don’t overlap).
-  **Reuse a Seat** – The same seat #5 may carry $A \rightarrow B$ for one person and $B \rightarrow D$ for another – both valid because they don’t overlap.
-  **Fast Availability** – Given a segment request, find a free seat efficiently – don’t scan every booking on every seat every time.

Example Use Case

- Seat 5 already has $A \rightarrow B$. A new $B \rightarrow D$ request $\rightarrow$ fits on seat 5 (no overlap).
- Seat 5 has $A \rightarrow C$. A new $B \rightarrow D$ request $\rightarrow$ does NOT fit ($B \rightarrow C$ overlaps), try another seat.

III. Booking Under Load & the Last-Seat Race

When two people try to grab the last usable seat for overlapping segments at the same instant, only one may win.

-  **Atomic Booking** – Two concurrent requests for the last free seat on overlapping legs must not both succeed.
-  **Correct Counting** – Availability must be checked per segment, not just as a route-wide “seats sold” count – that’s the bug candidates must avoid.

IV. Waitlist & Cancellations

When the bus is full for a segment, passengers join a waitlist. When someone cancels and frees part of the seat, promote the right person fairly.

-  **Waitlist Order** – Keep a fair order (first-come-first-served) for each segment.
-  **Smart Promotion** – When a cancellation frees seat #5 for A→C, promote a waitlisted passenger whose segment now fits – and explain how you pick among several eligible ones.

Outcome

- The bus is filled far better than a naive “one seat = one passenger” design, so more employees travel with fewer buses.



Points to Take Care Of (this is what we look for):

- Model a seat’s bookings as time/stop intervals – the whole system rests on getting the overlap test exactly right (watch the “just touching at a stop” boundary).
 - For speed, think about the data structure: sorting by stop, or an interval/segment tree, instead of $O(N)$ scans per request.
 - Handle the concurrency: last overlapping seat booked twice, or a cancel + waitlist promotion happening together, must never corrupt the seat map.
 - No-show, mid-route boarding, and a trip moved to a different bus overnight are all real edge cases – handle at least the first two.
-

